HDOC Day 41

Question: Write a menu-driven C program to read two paragraphs using separate functions. Read paragraphs, find the longest and shortest words, remove repeated spaces, convert to title case, and show case-insensitive word frequencies. Use pointers for output strings.

C Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define SIZE 1000

void readParagraph(char p[], size_t size) {
    fgets(p, (int)size, stdin);
    p[strcspn(p, "\n")] = '\0';
}

void findLongestAndShortestWords(const char p[], char longest[],
                                 char shortest[]) {
    char copy[SIZE];
    strcpy(copy, p);
    longest[0] = shortest[0] = '\0';
    char *word = strtok(copy, " \t.,!?;:\n");
    while (word != NULL) {
        if (strlen(longest) < strlen(word)) strcpy(longest, word);
        if (shortest[0] == '\0' || strlen(shortest) > strlen(word))
            strcpy(shortest, word);
        word = strtok(NULL, " \t.,!?;:\n");
    }
}

void removeRepeatedSpaces(const char p[], char result[]) {
    int i, j = 0, previousSpace = 0;
    for (i = 0; p[i] != '\0'; i++) {
        int space = isspace((unsigned char)p[i]);
        if (!space || !previousSpace) result[j++] = p[i];
        previousSpace = space;
    }
    result[j] = '\0';
}

void convertToTitleCase(const char p[], char result[]) {
    int i, newWord = 1;
    for (i = 0; p[i] != '\0'; i++) {
        unsigned char ch = (unsigned char)p[i];
        result[i] = newWord ? toupper(ch) : tolower(ch);
        if (isalpha(ch)) newWord = 0;
        else newWord = 1;
    }
    result[i] = '\0';
}

void findWordFrequencies(const char p[]) {
    char copy[SIZE], words[SIZE / 2][SIZE];
    int counts[SIZE / 2] = {0}, total = 0, i, j, found;
    strcpy(copy, p);
    char *word = strtok(copy, " \t.,!?;:\n");
    while (word != NULL) {
        for (i = 0; word[i]; i++)
            word[i] = tolower((unsigned char)word[i]);
        found = -1;
        for (j = 0; j < total; j++)
            if (strcmp(words[j], word) == 0) { found = j; break; }
        if (found >= 0) counts[found]++;
        else if (total < SIZE / 2) {
            strcpy(words[total], word);
            counts[total++] = 1;
```

```
        }
        word = strtok(NULL, " \t.,!?;:\n");
    }
    for (i = 0; i < total; i++)
        printf("%s: %d\n", words[i], counts[i]);
}

int main(void) {
    char p[2][SIZE], longest[SIZE], shortest[SIZE], result[SIZE];
    int choice, i;
    printf("Enter paragraph 1: "); readParagraph(p[0], SIZE);
    printf("Enter paragraph 2: "); readParagraph(p[1], SIZE);
    do {
        printf("\n1.Read again  2.Longest/shortest  3.Remove spaces\n");
        printf("4.Title case  5.Word frequency  6.Exit\nChoice: ");
        if (scanf("%d", &choice) != 1) break;
        while (getchar() != '\n');
        if (choice == 1) {
            for (i = 0; i < 2; i++) {
                printf("Enter paragraph %d: ", i + 1);
                readParagraph(p[i], SIZE);
            }
        } else if (choice >= 2 && choice <= 5) {
            for (i = 0; i < 2; i++) {
                printf("\nParagraph %d:\n", i + 1);
                switch (choice) {
                    case 2:
                        findLongestAndShortestWords(p[i], longest, shortest);
                        printf("Longest: %s\nShortest: %s\n", longest, shortest);
                        break;
                    case 3:
                        removeRepeatedSpaces(p[i], result);
                        printf("%s\n", result); break;
                    case 4:
                        convertToTitleCase(p[i], result);
                        printf("%s\n", result); break;
                    case 5: findWordFrequencies(p[i]); break;
                }
            }
        } else if (choice != 6) printf("Invalid choice\n");
    } while (choice != 6);
    return 0;
}
```

Sample Output:

```
Enter paragraph 1: Hello  hello world
Enter paragraph 2: C programming is fun
Choice: 2
Paragraph 1:
Longest: Hello
Shortest: world
Paragraph 2:
Longest: programming
Shortest: C
Choice: 5
Paragraph 1:
hello: 2
world: 1
Paragraph 2:
c: 1
programming: 1
is: 1
fun: 1
```